

Tutorial 4

Creational Patterns

1. How can I ensure that **only one instance of a configuration manager or a database connection pool exists** throughout the entire application, and provide a global access point to it? **Ans: Singleton**
2. I have a system that needs to create different types of documents (e.g., PDF, HTML, plain text), and **the exact type is determined at runtime**. How can I delegate the instantiation logic to subclasses? Ans:
3. I need to create **families of related objects** (e.g., a set of UI widgets like buttons, checkboxes, and text boxes, all belonging to the "Dark Theme" or "Light Theme"). How can I ensure that products from one family are never mixed with products from another?
4. A complex object, like a **Pizza**, has **many optional components** (crust type, sauce, toppings). How can I construct the object step-by-step and allow the construction process to produce different representations?
5. Creating new objects is computationally expensive, and I often need copies of existing objects. How can I **create new objects by copying an existing object (prototyping)** instead of creating them from scratch?

Structural Patterns

1. I'm integrating a new Legacy Library into my application, but its interface is **incompatible** with the rest of my code. How can I make its interface compatible without changing its source code?
2. How can I **dynamically add new behaviors or responsibilities to an object** without altering its structure or affecting the behavior of other objects of the same class (e.g., adding logging, encryption, or compression to a data stream)?
3. I need to treat a **group of objects** (like a folder containing files and sub-folders) the **same way** I treat an individual object (like a single file). How can I create a tree-like structure and work with it uniformly?
4. How can I provide a **placeholder or a surrogate** for another object to control access to it (e.g., a **Virtual Proxy** to delay loading a large image until it's needed, or a **Security Proxy** to check access permissions)?
5. The subsystem responsible for Order Processing is overly complex, involving many classes and interfaces. How can I provide a **simplified, unified interface** to this complex subsystem for clients?

Behavioral Patterns

1. When the **state of one object (Subject)** changes (e.g., a stock price update), how can I automatically notify and update all its dependents (Observers) without coupling them too tightly?
2. I have several different **algorithms** for Sorting Data (Quick Sort, Merge Sort, Bubble Sort), and I need to be able to **switch between them dynamically** at runtime. How should I structure this?

3. I need to implement **undoable operations** (like a text editor's undo feature) or **parameterize objects with an action to perform**. How can I encapsulate a request as an object?
4. How can I **access the elements of a collection** (like a list, tree, or graph) sequentially **without exposing its underlying representation**?
5. The behavior of an object, such as a **Traffic\ Light**, changes drastically based on its Internal State (Red, Yellow, Green). How can I manage these state-specific behaviors cleanly?
6. I have an algorithm for Generating Reports that has a fixed overall structure, but certain **steps need to be implemented differently by subclasses**. How can I define the skeleton of an algorithm in an operation, deferring some steps to subclasses?
7. A Request (e.g., a message or a loan application) needs to be passed through a **series of potential processors** until one of them handles it. How can I decouple the sender from the receivers?
8. I have many objects (e.g., components in a Dialog Box) that need to **communicate and coordinate**, but they have become tightly coupled. How can I centralize their communication to reduce complexity?
9. I need to be able to **save and restore the internal state of an object** (Originator) without violating encapsulation. For example, saving the current state of a game.